

MODULE 5

Microservices and Data

Managing persistence, transactions, consistency and caching.

SQL integration
with Spring
Boot

Data ownership

Persistence
patterns

Local
transactions

Distributed
transactions

Saga pattern

Redis caching

LEARNING OBJECTIVES

SQL integration, persistence patterns, transaction strategies and Redis caching.

BY THE END OF THIS MODULE, YOU CAN

1. Integrate Spring Boot with SQL
2. Explain service-owned data
3. Use Spring Data JPA repositories
4. Understand persistence patterns
5. Implement local transactions
6. Explain distributed transaction challenges
7. Describe Saga patterns
8. Understand compensating transactions
9. Explain Redis caching
10. Design cache expiration and invalidation

FOUR PARTS

1 SQL persistence

Spring Boot

Repository

SQL Database

2 Data ownership

Microservice

Owns its data

3 Transactions

Local ACID transaction

vs.

Distributed business transaction

4 Caching

Application

Redis

Database

KEY TAKEAWAY Data is where moving from a monolith to microservices changes architecture the most.

FROM MODULE 4 TO MODULE 5

Services now talk to each other — this module is about the data behind each one.

MODULE 4 GAVE US

- ✓ REST and events between services
- ✓ Brokers, topics and consumer groups
- ✓ Idempotent consumers and dead letters
- ✓ Choreography and orchestration
- ✓ The transactional outbox

NOW Give each service its own data — and keep the business outcome correct.



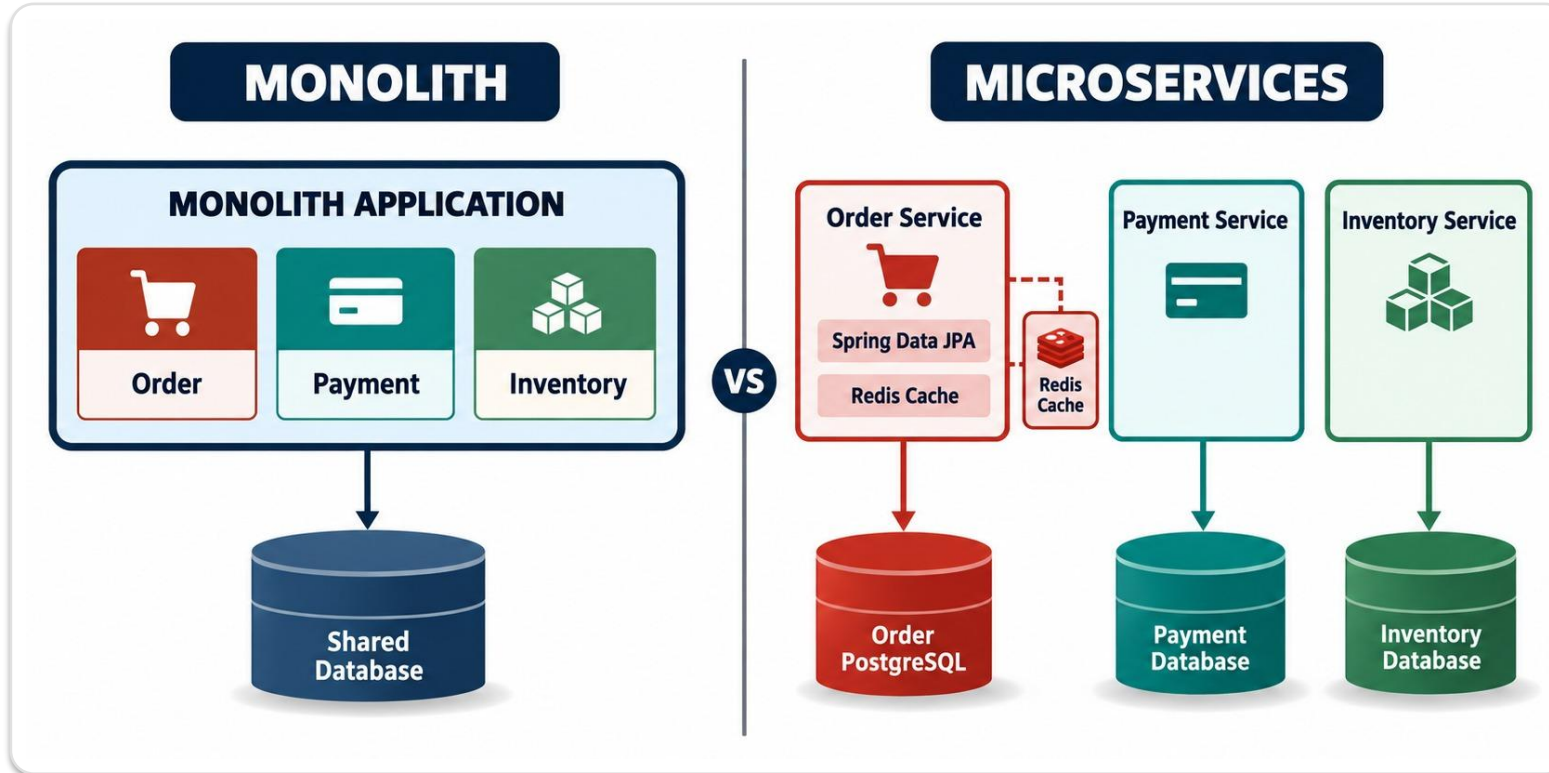
OUR PATH THROUGH MODULE 5

- 1 Persist an order with Spring Data JPA [Part 1](#)
- 2 Configure the DataSource and migrations [Part 1](#)
- 3 Keep each service's data private [Part 2](#)
- 4 Get data you don't own [Part 2](#)
- 5 Commit locally; coordinate with Sagas [Part 3](#)
- 6 Compensate when a step fails [Part 3](#)
- 7 Cache reads with Redis, safely [Part 4](#)

KEY TAKEAWAY Module 4 connected services; Module 5 makes each one own its data and stay correct.

WHY DATA MANAGEMENT CHANGES WITH MICROSERVICES

From one shared database toward independently owned service data.



NOW YOU MUST HANDLE

- ! Distributed state
- ! Eventual consistency
- ! Cross-service workflows
- ! Failure recovery

One business operation

Placing an order may now touch the Order, Payment and Inventory databases.

FOUR WORDS Ownership, autonomy, consistency, distribution.

KEY TAKEAWAY Microservices gain autonomy, but one business operation may now cross several databases.

PART 1: SQL PERSISTENCE WITH SPRING BOOT

Before sharing data across services, store it well inside one.

Part 1
SQL persistence

Part 2
Data ownership

Part 3
Transactions

Part 4
Caching

SO FAR

- ✓ Services integrate through REST and events
- ✓ Consumers are idempotent; failures dead-letter
- ✓ An outbox publishes events safely

NEXT QUESTION

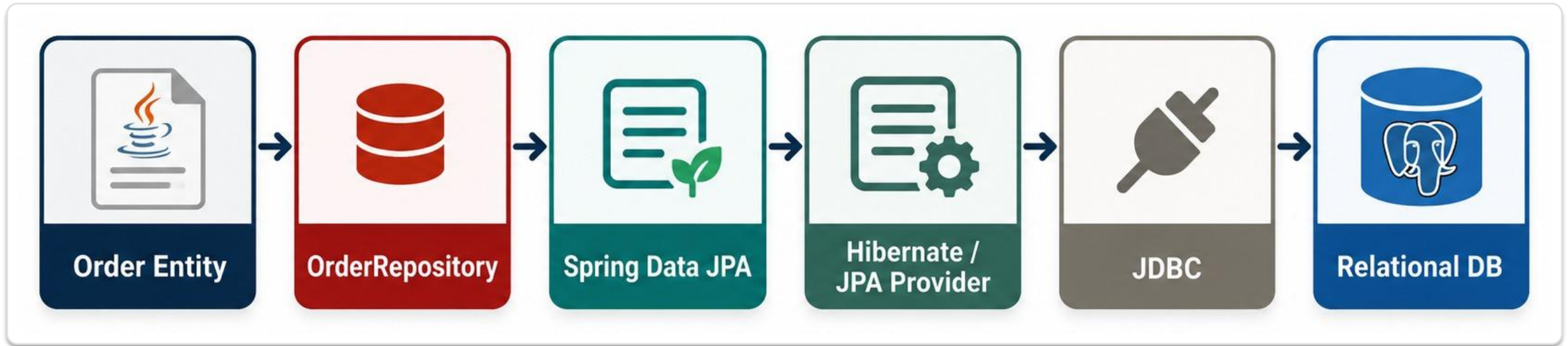
How does a Spring Boot service store and read its own data correctly?

IN THIS PART

- 1 The persistence stack: JDBC, JPA, Hibernate, Spring Data
- 2 DataSource configuration and Flyway migrations
- 3 Mapping entities — and references across services
- 4 Repositories and the end-to-end flow
- 5 Exercise: spot the persistence problems

SPRING BOOT SQL INTEGRATION ARCHITECTURE

Entity → repository → Spring Data JPA → Hibernate → JDBC → SQL database.



JDBC

The low-level Java API that talks to the database.

JPA

The specification for mapping Java objects to tables.

Hibernate

The JPA implementation Spring Boot uses by default.

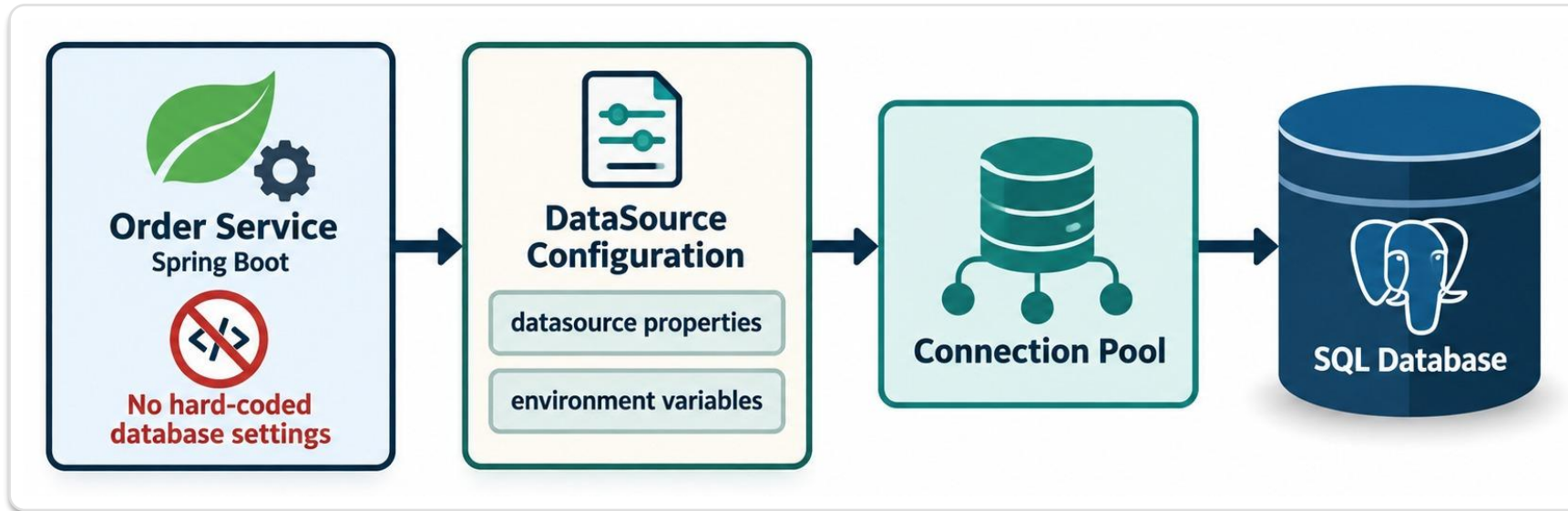
Spring Data JPA

Generates repository implementations at runtime.

KEY TAKEAWAY JDBC talks to the database, JPA and Hibernate map objects, and Spring Data removes the plumbing.

CONFIGURING A SQL DATASOURCE

Spring Boot datasource configuration and connection properties.



✓ application.yml

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
```

TYPICAL VALUES

- ✓ JDBC URL
- ✓ Username and password
- ✓ Driver
- ✓ Pool settings

Connection pool

HikariCP reuses connections instead of opening one per request.

SECRETS From the environment — later, OpenShift Secrets.

KEY TAKEAWAY Never commit database credentials — reference them and let the environment supply them.

SCHEMA MIGRATIONS WITH FLYWAY

Versioned SQL files own the schema; Hibernate only checks it.

db/migration/V1__create_orders.sql

```
CREATE TABLE orders (  
  id          BIGSERIAL      PRIMARY KEY,  
  customer_id VARCHAR(20)    NOT NULL,  
  status      VARCHAR(20)    NOT NULL,  
  total       NUMERIC(12, 2) NOT NULL,  
  version     BIGINT         NOT NULL DEFAULT 0,  
  created_at  TIMESTAMP      NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_orders_customer ON orders (customer_id);
```

✓ application.yml

```
spring:  
  jpa:  
    hibernate:  
      ddl-auto: validate      # Flyway owns the schema  
  flyway:  
    locations: classpath:db/migration # the default  
  datasource:  
    hikari:  
      maximum-pool-size: 10  
      connection-timeout: 3000 # milliseconds
```

Versioned

V1, V2 ... run once, in order, recorded in flyway_schema_history.

validate

Hibernate checks the entities match — it never changes tables.

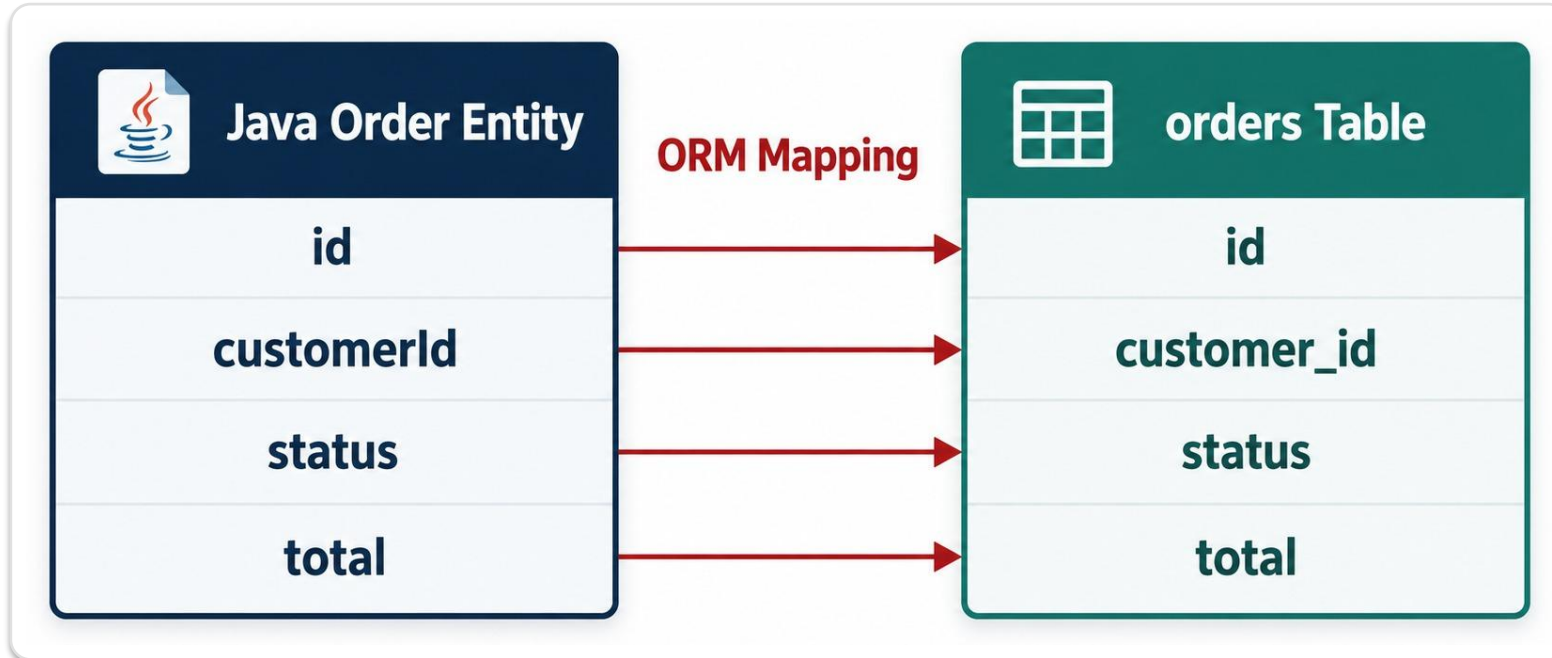
Never edit

an applied migration. Add V2 instead.

KEY TAKEAWAY Let Flyway create and evolve the schema; let Hibernate validate it — never ddl-auto: update in production.

MAPPING JAVA OBJECTS TO RELATIONAL DATA

@Entity, @Id, columns, tables and object-relational mapping.



Order.java

```
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue
    private Long id;
    private String customerId;
    private String status;
    private BigDecimal total;
}
```

Naming

Spring Boot maps `customerId` to `customer_id` automatically.

ORM ≠ NO SQL You still need keys, indexes and query awareness.

KEY TAKEAWAY ORM maps Java objects to tables — production developers understand both models.

RELATIONSHIPS INSIDE A SERVICE, IDS ACROSS SERVICES

Map what shares a lifecycle and a database; reference everything else by ID.

✓ Inside Order Service — map it

```
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();

    private String customerId; // owned by Customer Service
}
```

✗ Across services — don't map it

```
@Entity
public class Order {

    @ManyToOne // Customer lives in
    private Customer customer; // another service's DB

    @ManyToOne // a join into
    private Payment payment; // Payment's tables
}
```

Inside one service

Order and OrderItem share a lifecycle and a database — map them.

Across services

Store only the ID; call Customer Service when you need details.

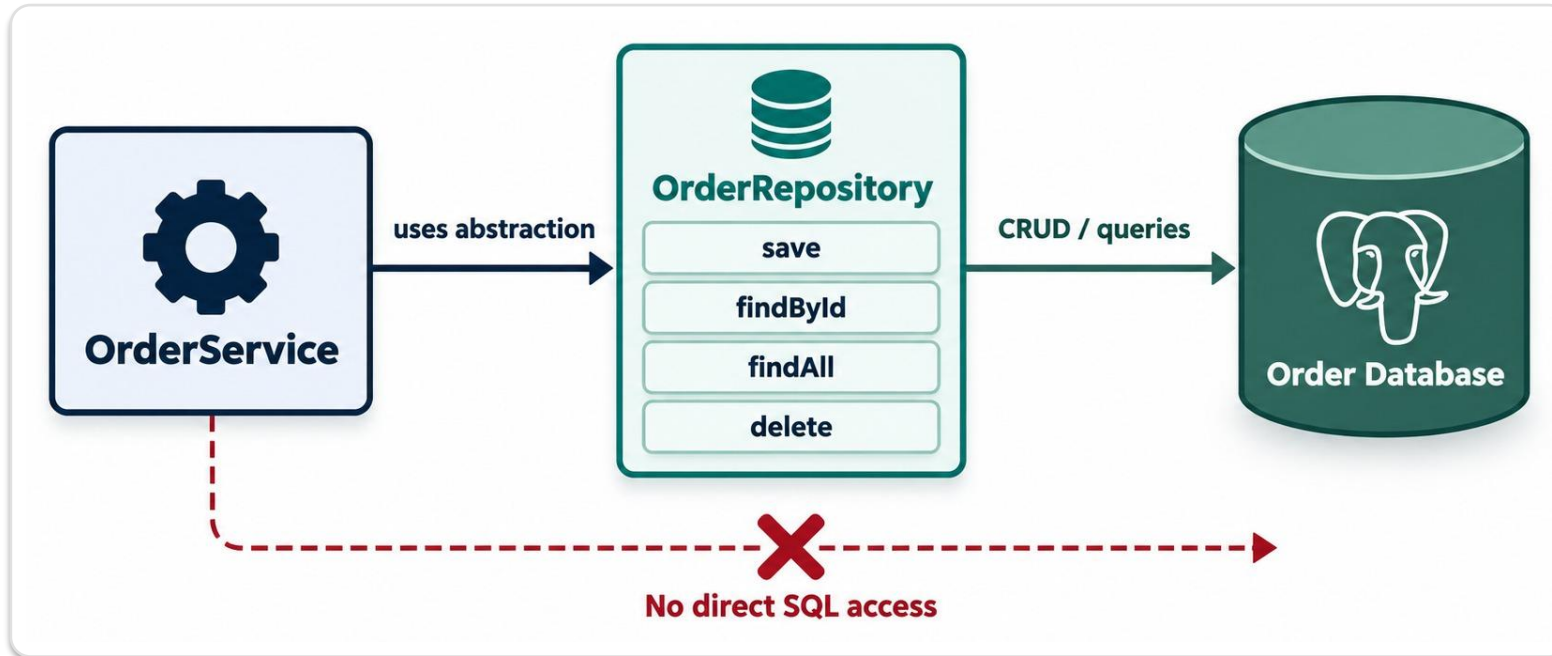
LAZY by default

Load collections only when needed — watch for N+1 queries.

KEY TAKEAWAY JPA relationships stop at the service boundary — across it, keep an ID and use the owner's API or events.

REPOSITORY PATTERN WITH SPRING DATA JPA

JpaRepository, CRUD operations and query abstraction.



DERIVED QUERY METHODS

findByStatus

findByCustomerId

existsByIdAndStatus

countByCustomerId

When a name gets too long

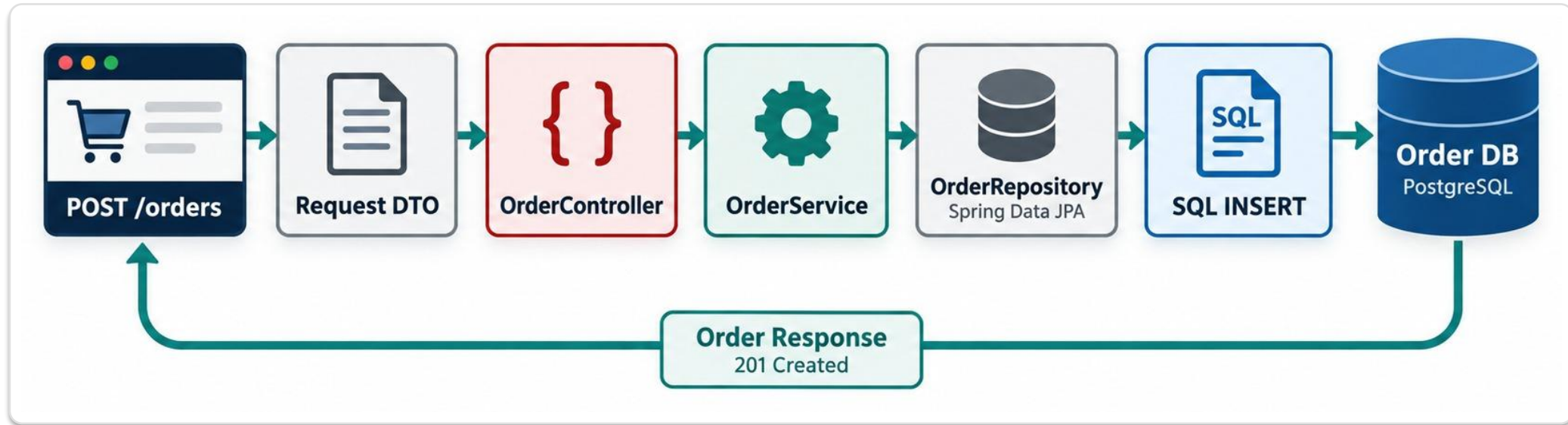
```
@Query("select o from Order o "
      + "where o.total > :min")
List<Order> bigOrders(
    @Param("min") BigDecimal min);
```

KEEP IT CLEAN Queries in the repository;
business rules in the service.

KEY TAKEAWAY Repositories give CRUD and derived queries — business logic stays in the service.

END-TO-END PERSISTENCE FLOW

REST request → controller → service → repository → SQL → response.



Request body

```
{
  "customerId": "CUST-501",
  "total": 199.99
}
```

✓ The key line

```
Order saved = orders.save(order);
// INSERT INTO orders (...) VALUES (...)
```

CONNECTS REST from Module 3 with SQL persistence from this module.

KEY TAKEAWAY One request travels from JSON to a SQL INSERT and back as a 201 Created response.

EXERCISE: SPOT THE PERSISTENCE PROBLEMS

Review a pull request for Order Service.

PRACTICE EXERCISE

12 MIN

SCENARIO

The pull request contains:

```
# application.yml
spring.jpa.hibernate.ddl-auto: update
spring.datasource.password: Prod#2026

@ManyToOne
private Customer customer;

@GetMapping("/orders")
List<Order> all() {
    return orders.findAll();
}
```

YOUR TASKS

- 1 Find at least four problems
- 2 Say why each one hurts in production
- 3 Propose a fix for each

EXPECTED OUTCOME

- ✓ ddl-auto update → Flyway + validate
- ✓ Password in YAML → environment secret
- ✓ @ManyToOne Customer → store customerId
- ✓ Returns entities → return DTOs
- ✓ findAll, no paging → Pageable

KEY TAKEAWAY Schema by migration, secrets from the environment, IDs across services, DTOs out, and pages for lists.

PART 2: DATA OWNERSHIP

Each service's data is private — like its code.



SO FAR

- ✓ Spring Data JPA maps entities and generates repositories
- ✓ Flyway owns the schema; secrets come from the environment
- ✓ One request becomes SQL and a DTO

NEXT QUESTION

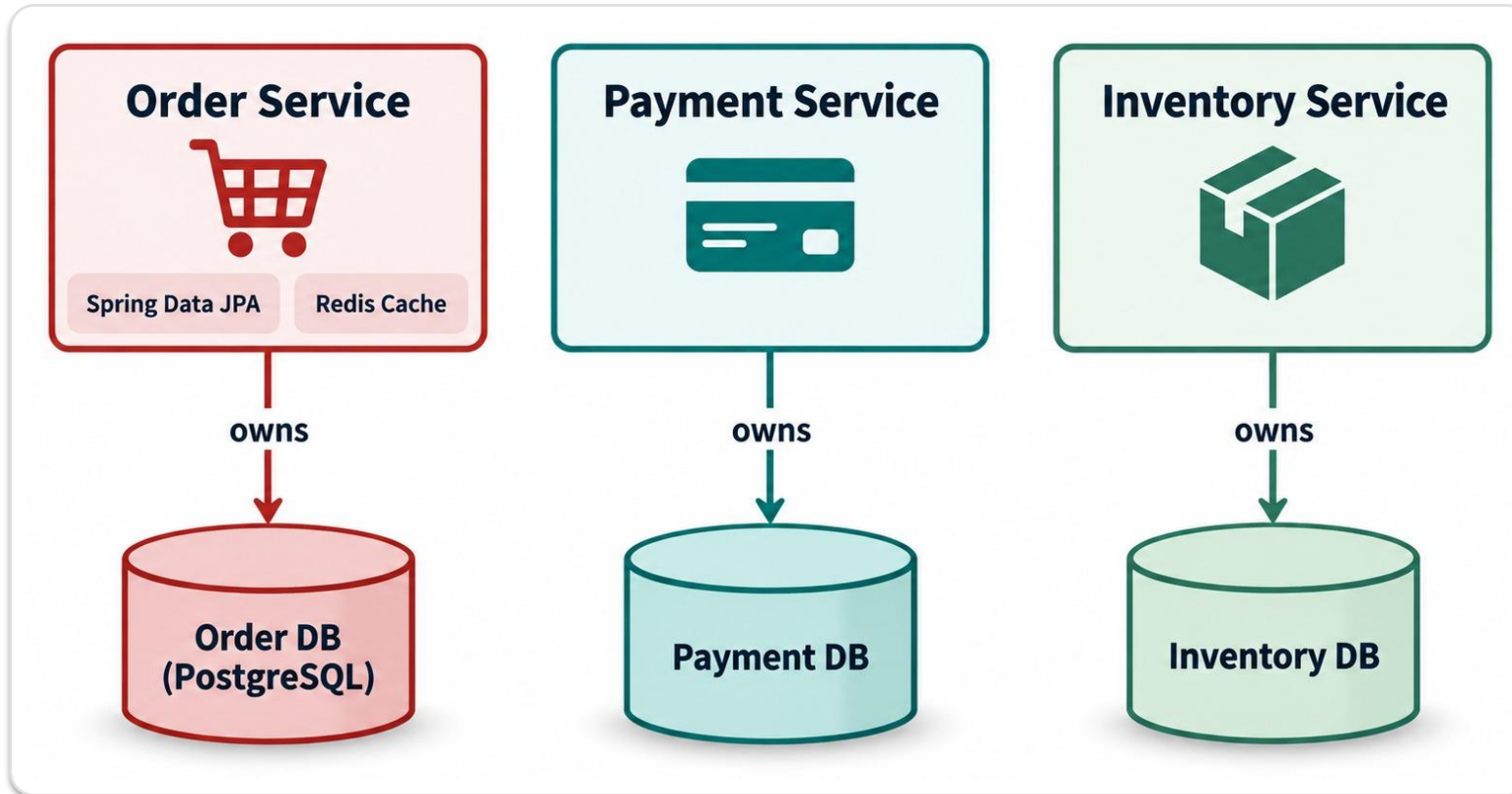
Who is allowed to touch which data — and how do others get it?

IN THIS PART

- 1 Database per service — and its trade-offs
- 2 Why shared database access is dangerous
- 3 Getting data you don't own
- 4 Polyglot persistence
- 5 Exercise 2.1: map data ownership

DATABASE PER SERVICE PATTERN

Each microservice owns and controls its persistence boundary.



YOU GAIN

- ✓ Independent schema changes
- ✓ Independent deployment
- ✓ Clear ownership

YOU GIVE UP

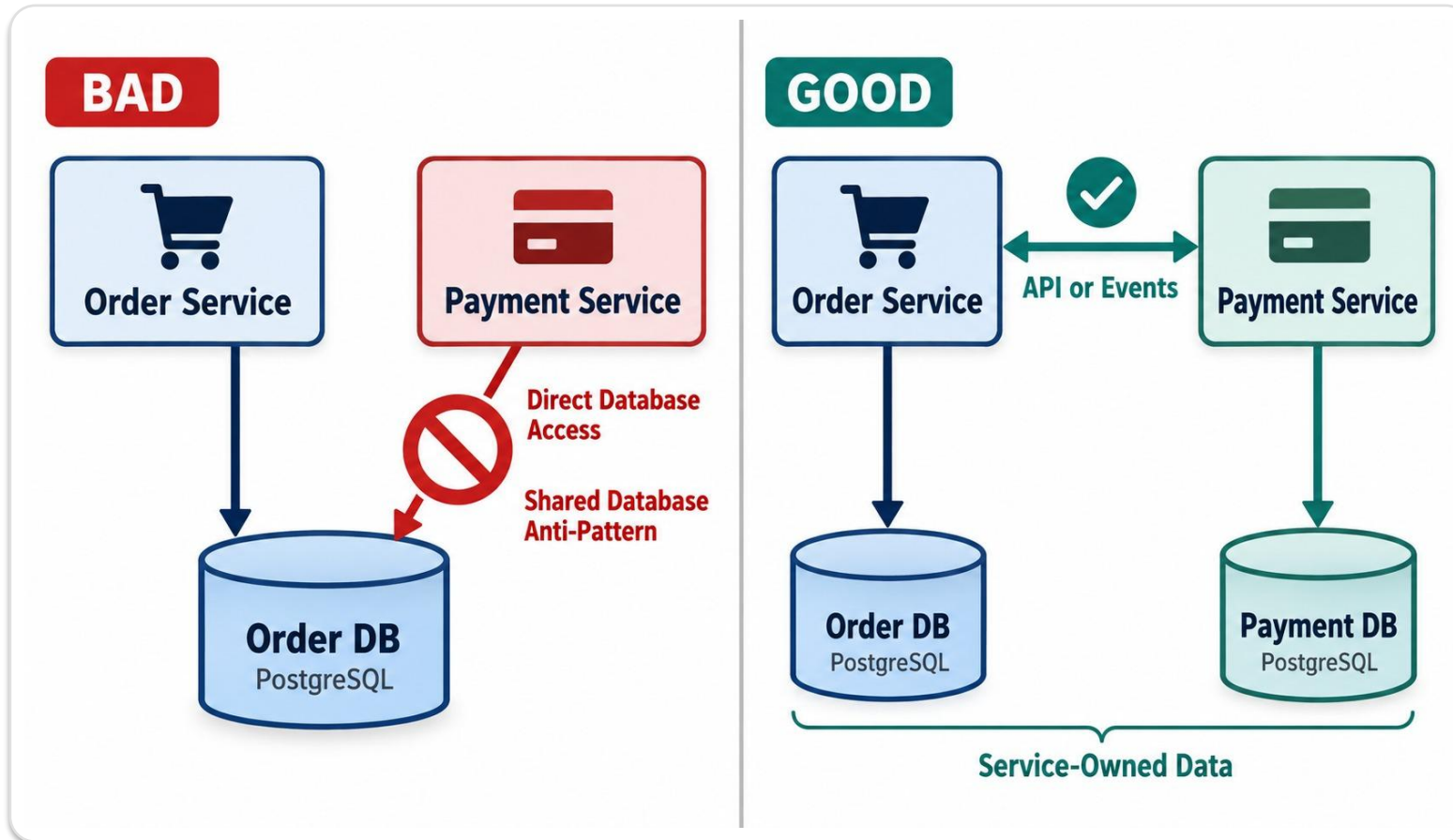
- ✗ Easy cross-table joins
- ✗ One ACID transaction for everything

OWNERSHIP, NOT SERVERS Separate schemas on one server can still work.

KEY TAKEAWAY A service owns its data — others go through its API or events, never its tables.

WHY SHARED DATABASE ACCESS IS DANGEROUS

One service reading or writing another service's tables creates coupling.



✗ Payment writes Order's table

```
UPDATE orders
SET status = 'PAID'
WHERE id = 1001;  -- run by Payment
```

PAYMENT NOW DEPENDS ON

- ! Order's table structure
- ! Valid status values
- ! Every future schema change

BETTER Payment publishes PaymentAuthorized; Order updates its own row.

KEY TAKEAWAY Data ownership is part of the microservice contract boundary.

GETTING DATA YOU DON'T OWN

API calls, API composition and event-built read models — never a cross-service join.

Approach	How it works	Trade-off
REST API call	Ask the owner at request time	Fresh, but a runtime dependency
API composition	Combine several owners' APIs	Many calls per request
Read model (CQRS)	A local copy built from events	Fast, but eventually consistent
Shared table or join	Query another service's database	✗ Breaks ownership — never

✓ **Order keeps a local customer view**

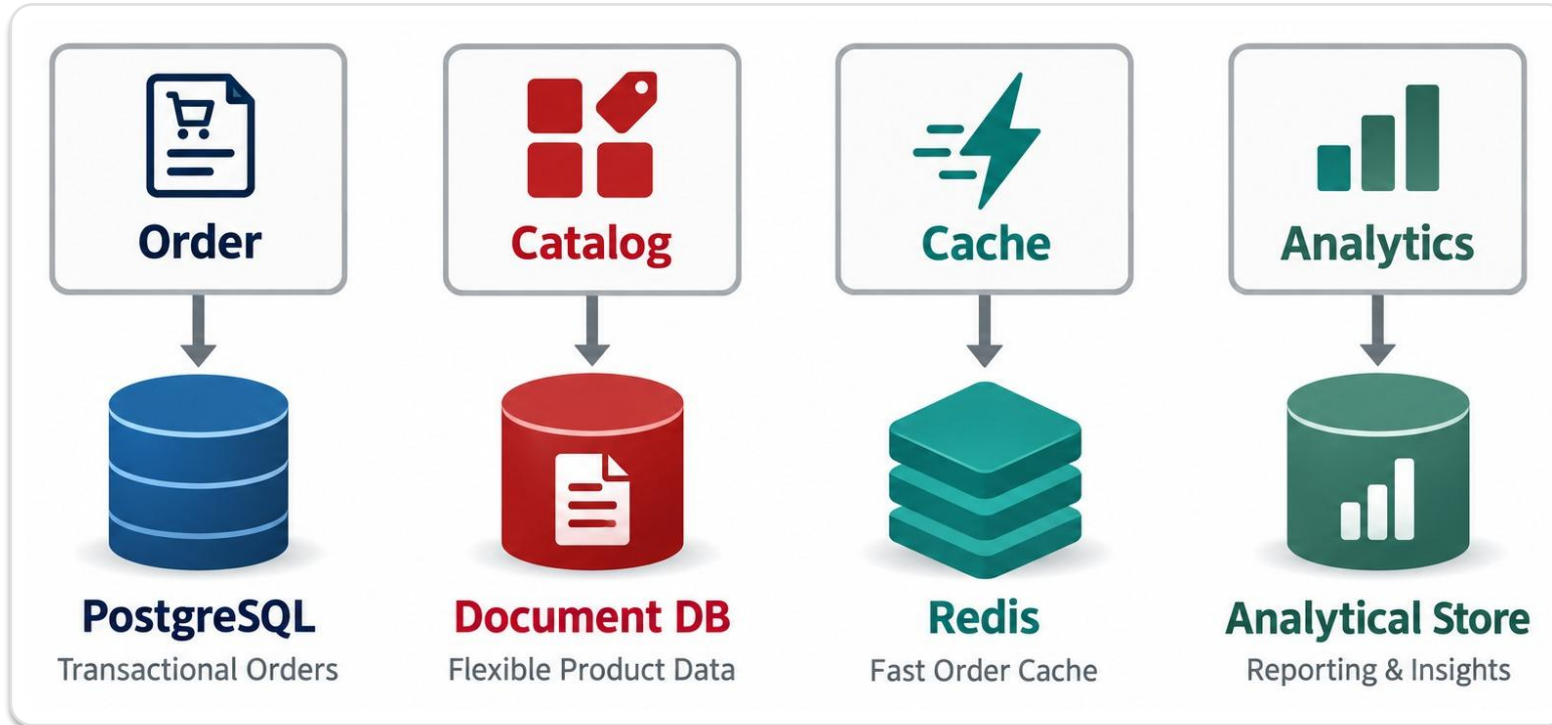
```
@KafkaListener(topics = "customer.events")
void on(CustomerUpdated e) {
    views.save(new CustomerView(
        e.customerId(), e.name()));
}
```

RULE Copy only the fields you need; the owner stays the source of truth.

KEY TAKEAWAY Get other services' data through their APIs or their events — the owner stays the source of truth.

POLYGLOT PERSISTENCE

Services choose SQL, document, key-value or other stores based on requirements.



EVERY NEW STORE NEEDS

Operations skills

Monitoring

Backups

Security patching

Disaster recovery

Developer expertise

In this course

Every lab service uses PostgreSQL. Redis and document stores are lecture-only.

RULE Add a technology only when the benefit justifies the cost.

KEY TAKEAWAY Pick the right store per service — but only when the benefit outweighs the operational cost.

EXERCISE 2.1 — MAP ACCOUNT VS. TRANSACTION DATA

Day 2 checkpoint A: separate databases and no cross-service tables.

OFFICIAL CHECKPOINT A

10 MIN

curriculum/day-02/exercises/exercise-2.1-data-ownership.md

SCENARIO

Day 2 splits the banking platform into Account Service (Lab 1) and Transaction Service (Lab 2).

Scope: Account, Transaction and Risk Assessment only.

YOUR TASKS

- 1 Account table lives only in account_db; transaction table only in transaction_db
- 2 Transaction stores accountId as a reference, not a cross-service foreign key
- 3 Eligibility is a REST GET to Account Service, not a SQL join
- 4 Redis is lecture-only — not part of this capstone

EXPECTED OUTCOME

- ✓ A two-column ownership map
- ✓ Zero shared tables
- ✓ accountId kept as a plain reference
- ✓ Eligibility checked over REST

KEY TAKEAWAY A two-column ownership map with zero shared tables.

PART 3: TRANSACTIONS AND SAGAS

All-or-nothing inside a service; coordinated steps across services.



SO FAR

- ✓ Each service owns its database
- ✓ Others use APIs, events or read models
- ✓ Extra stores only when justified

NEXT QUESTION

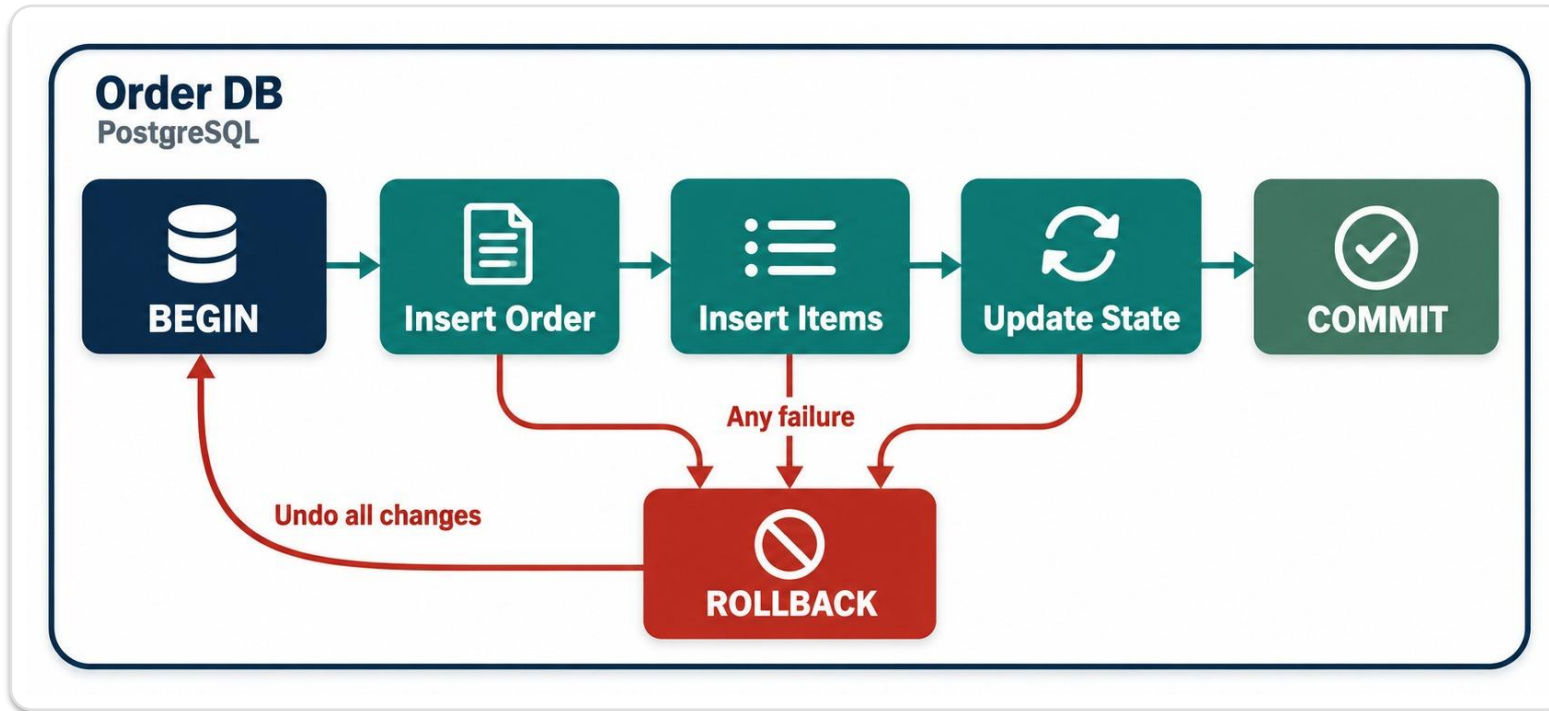
If one order touches three databases, how do we keep the business outcome correct?

IN THIS PART

- 1 Local transactions and optimistic locking
- 2 The distributed transaction problem
- 3 Why ACID stops at the service boundary
- 4 Sagas: choreography and orchestration
- 5 Compensating transactions, in code
- 6 Exercise: design the compensation

LOCAL TRANSACTIONS WITH @TRANSACTIONAL

Atomic operations inside a single Spring Boot service and database.



✓ One service, one database

```
@Transactional
public void create(CreateOrderRequest r) {
    Order order = orders.save(Order.from(r));
    items.saveAll(OrderItem.from(r, order));
}
```

RULES OF THUMB

- ✓ Put it on service methods
- ✓ Keep it short
- ✓ No remote calls inside

ROLLBACK RULE Runtime exceptions roll back by default; checked ones don't.

KEY TAKEAWAY @Transactional makes a service's local changes all-or-nothing.

CONCURRENT UPDATES: OPTIMISTIC LOCKING

Two users change the same order — only one update may win.

Order.java

```
@Entity
public class Order {
    @Id @GeneratedValue
    private Long id;

    @Version
    private long version;    // checked on every update

    private String status;
}
```

✓ Service and error handler

```
@Transactional
public OrderResponse cancel(Long id) {
    Order order = orders.findById(id)
        .orElseThrow(() -> new OrderNotFoundException(id));
    order.cancel();
    return OrderResponse.from(order);
} // UPDATE ... WHERE id = ? AND version = ?

@ExceptionHandler(ObjectOptimisticLockingFailureException.class)
@ResponseStatus(HttpStatus.CONFLICT)
ErrorResponse conflict() { ... } // 409: reload, retry
```

@Version

Hibernate adds the version to every UPDATE's WHERE clause.

Conflict

If someone changed the row first, the update matches nothing.

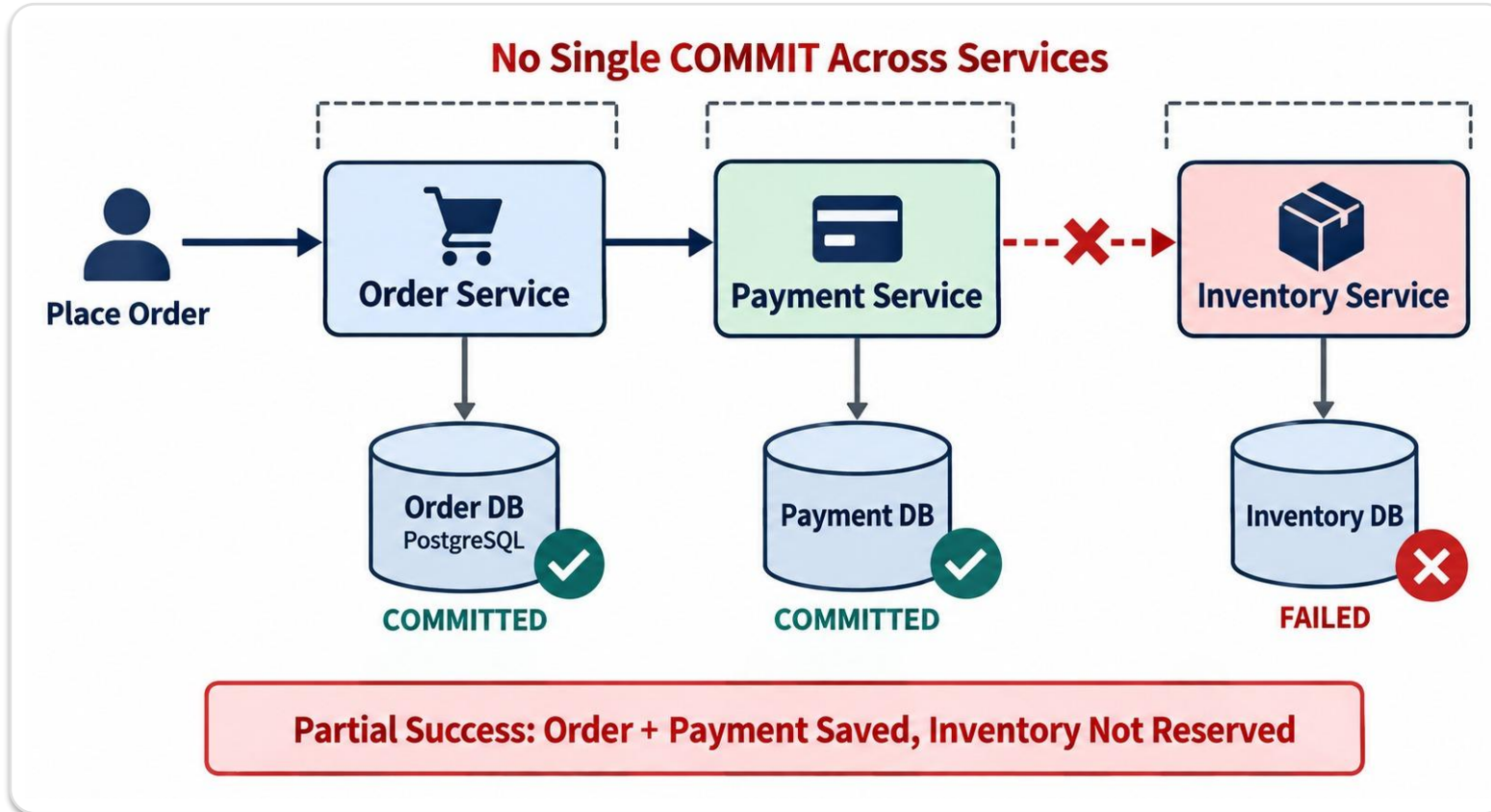
Response

Return 409 and let the client reload — no lost updates.

KEY TAKEAWAY A @Version column turns silent lost updates into a clear 409 Conflict.

THE DISTRIBUTED TRANSACTION PROBLEM

One business operation spanning Order, Payment and Inventory databases.



WHY IT'S HARD

- ! No single transaction manager
- ! Networks fail mid-way
- ! Payment providers can't roll back

Two-phase commit?

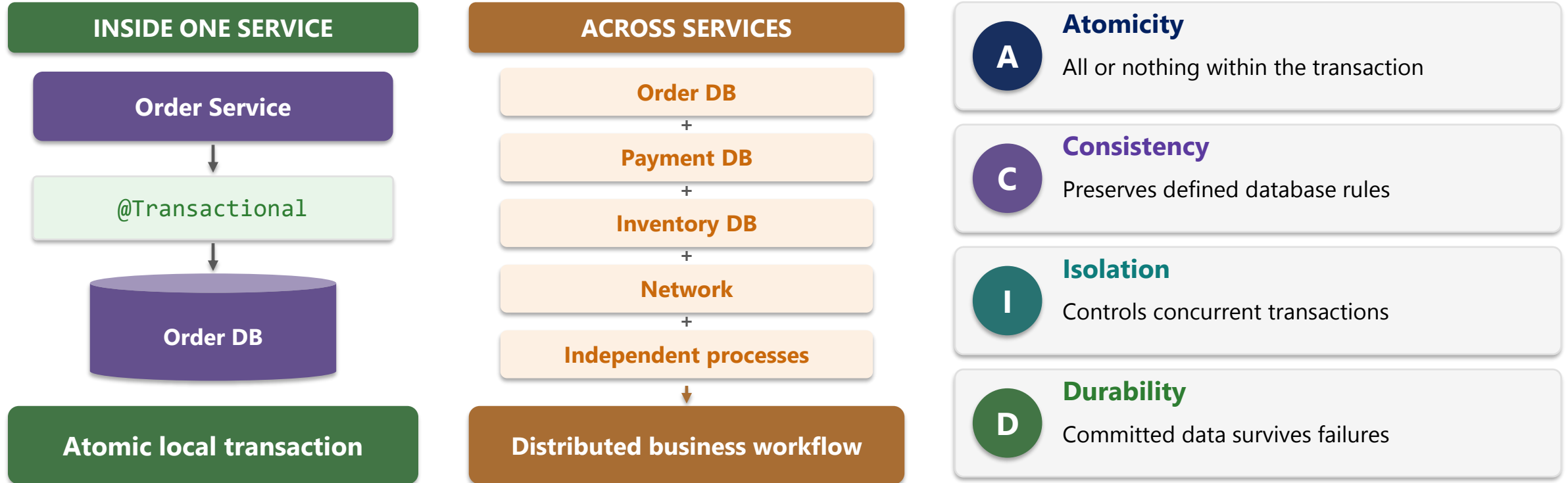
A coordinator locks every participant until all agree. It blocks, scales poorly and breaks on network splits.

THE QUESTION Who commits or undoes the whole business operation?

KEY TAKEAWAY When one business operation spans several databases, no single local transaction can undo it all.

WHY TRADITIONAL ACID TRANSACTIONS DON'T SPAN MICROSERVICES EASILY

Local transaction boundary versus distributed business workflow.

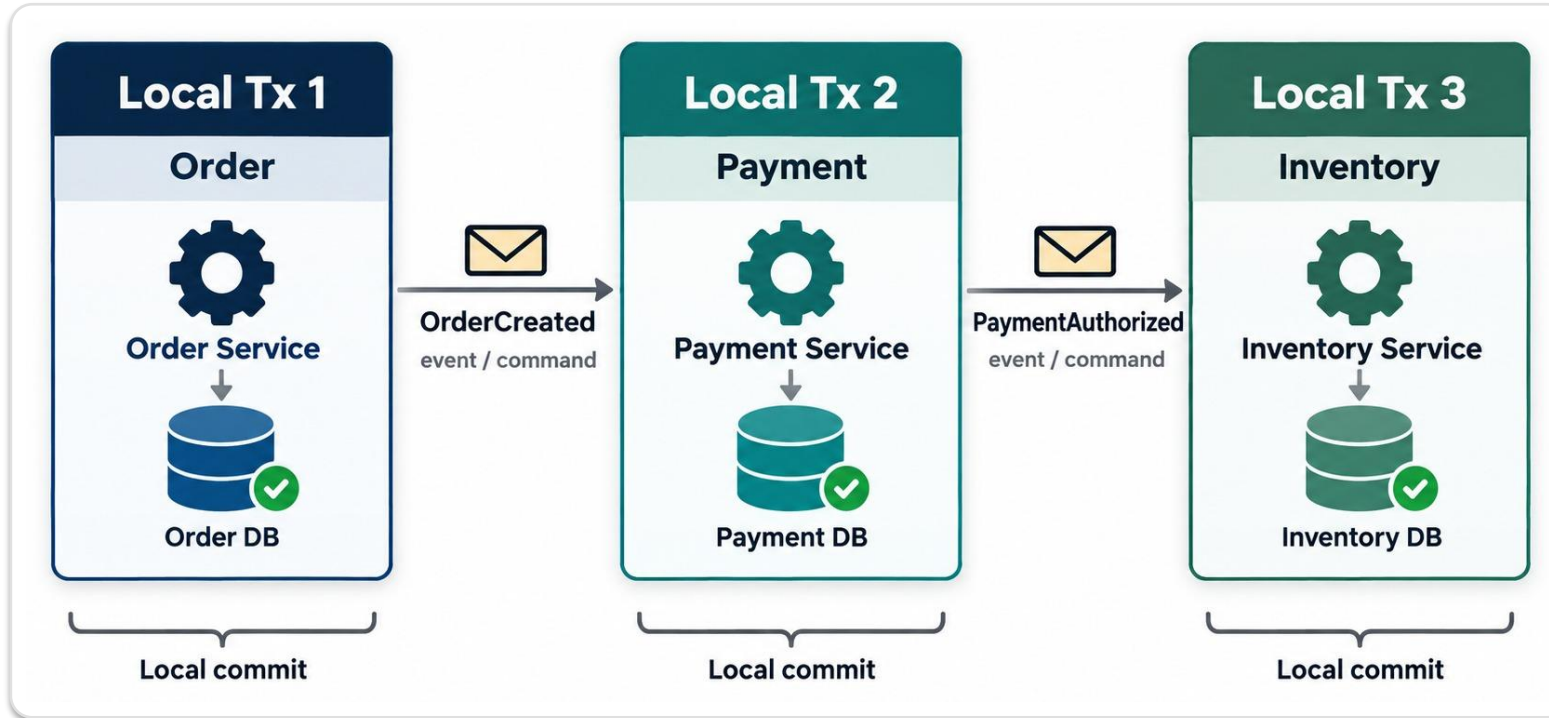


One large technical transaction → a sequence of local transactions coordinated as a business workflow → Saga

KEY TAKEAWAY ACID works inside one database; across services we coordinate local transactions instead.

SAGA PATTERN

A sequence of local transactions coordinated through commands or events.



Saga =

A sequence of local transactions + coordination + compensation.

Between steps

Each commit publishes an event or sends a command that starts the next step.

Visible in-between states

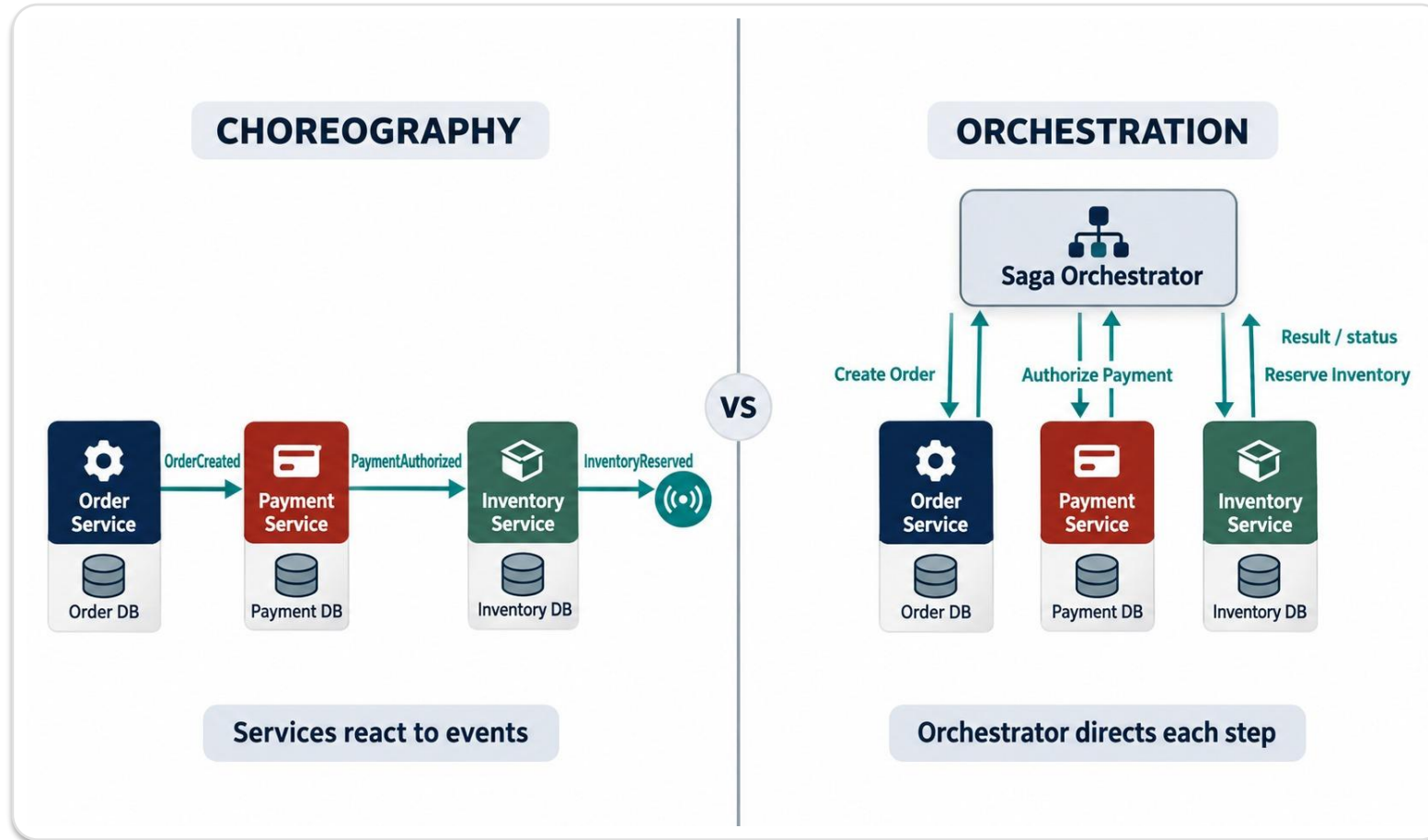
Others may see an order as PENDING — design screens and rules for it.

NOT ACID A Saga coordinates small transactions — it never makes one big one.

KEY TAKEAWAY A Saga breaks one distributed business transaction into local transactions that each commit.

SAGA CHOREOGRAPHY VS. SAGA ORCHESTRATION

Events trigger each local transaction, or an orchestrator commands them.



Choreography saga

Each local commit publishes the event that triggers the next step. Suits a few steps.

Orchestration saga

An orchestrator sends commands and stores saga state. Suits many steps and compensation.

EITHER WAY

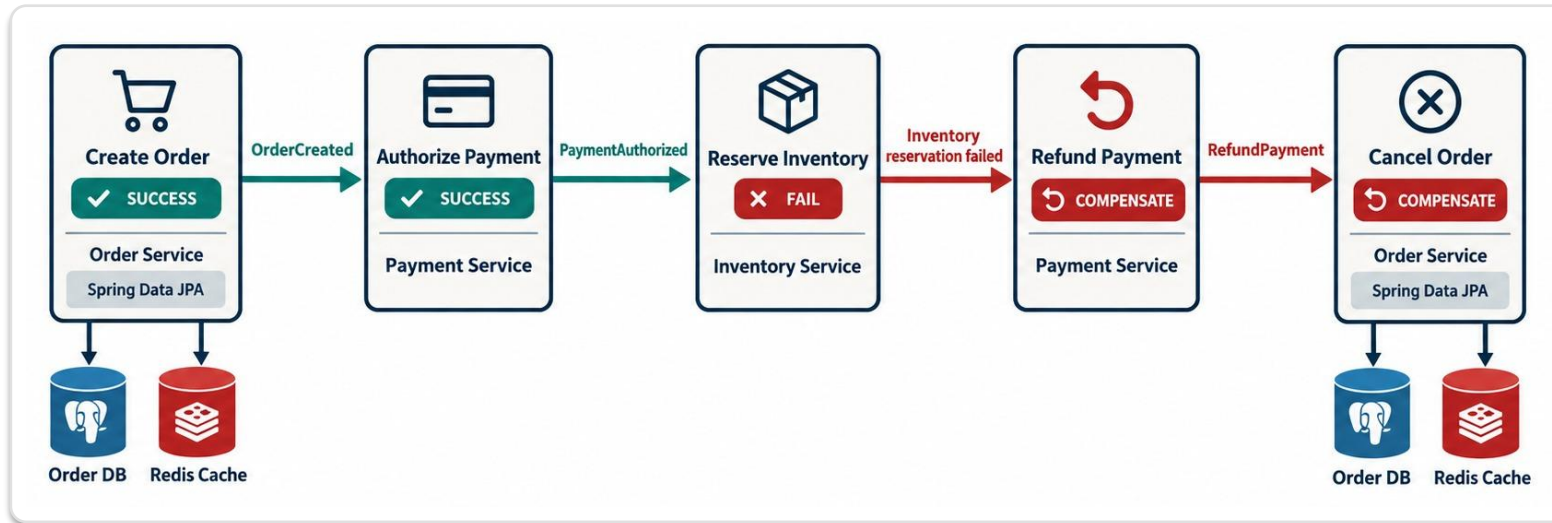
- ✓ Store saga state durably
- ✓ Make every step idempotent
- ✓ Trace with a correlation ID

FROM MODULE 4 Same styles — now each step is a local transaction.

KEY TAKEAWAY Choreography chains local transactions through events; orchestration commands them from one place.

COMPENSATING TRANSACTIONS AND FAILURE RECOVERY

Undo completed business actions when a later step fails.



EVERY STEP NEEDS ITS UNDO

- ✓ Charge \$200 → refund \$200
- ✓ Create order → cancel order
- ✓ Reserve stock → release stock

Refund ≠ rollback

It's a new business transaction, with its own record and audit trail.

COMPENSATIONS FAIL TOO Retry them, make them idempotent, alert on failure.

KEY TAKEAWAY Sagas don't roll back committed work — they run compensating business actions.

COMPENSATION IN CODE

Inventory failed: Order asks for a refund, Payment refunds exactly once.

Order Service — start compensating

```
@KafkaListener(topics = "inventory.reservation-failed",
               groupId = "order-service")
@Transactional
public void on(InventoryReservationFailed e) {
    Order order = orders.findById(e.orderId()).orElseThrow();
    if (order.isCancelled()) return;           // idempotent
    order.markCancelling();
    outbox.save(OutboxEvent.of("payments.refund-requested",
                              order.getId(),
                              new RefundPayment(order.getId(), order.getTotal())));
}
```

✓ Payment Service — compensate once

```
@KafkaListener(topics = "payments.refund-requested",
               groupId = "payment-service")
@Transactional
public void on(RefundPayment cmd) {
    if (refunds.existsByOrderId(cmd.orderId())) return;
    refunds.save(Refund.of(cmd.orderId(), cmd.amount()));
    outbox.save(OutboxEvent.of("payments.refunded",
                              cmd.orderId(), new PaymentRefunded(cmd.orderId())));
}
```

Command, then event

Order asks for a refund; Payment reports PaymentRefunded.

Idempotent both sides

A repeated message never refunds or cancels twice.

Outbox

State change and message commit together — from Module 4.

KEY TAKEAWAY Compensation is ordinary, idempotent business code — triggered by a failure event and published through the outbox.

EXERCISE: DESIGN THE COMPENSATION

A late step fails — decide how the saga recovers.

PRACTICE EXERCISE

12 MIN

SCENARIO

The order saga runs:
Create Order → Authorize Payment →
Reserve Inventory → Create Shipment

Shipping rejects the address after payment
and inventory both succeeded.

YOUR TASKS

- 1 List the compensations, in order
- 2 Mark which steps must be idempotent — and why
- 3 Say what the customer sees at each stage

EXPECTED OUTCOME

- ✓ Release the reserved inventory
- ✓ Refund or void the payment
- ✓ Mark the order CANCELLED — or ask for a new address
- ✓ Every compensation is idempotent
- ✓ Customer sees PENDING, then a clear notice

KEY TAKEAWAY Compensate in reverse order, make every step idempotent, and keep the customer informed.

PART 4: CACHING WITH REDIS

Faster reads — at the cost of a second copy of the data.



SO FAR

- ✓ Local transactions inside one service
- ✓ Sagas coordinate steps across services
- ✓ Compensations undo committed work

NEXT QUESTION

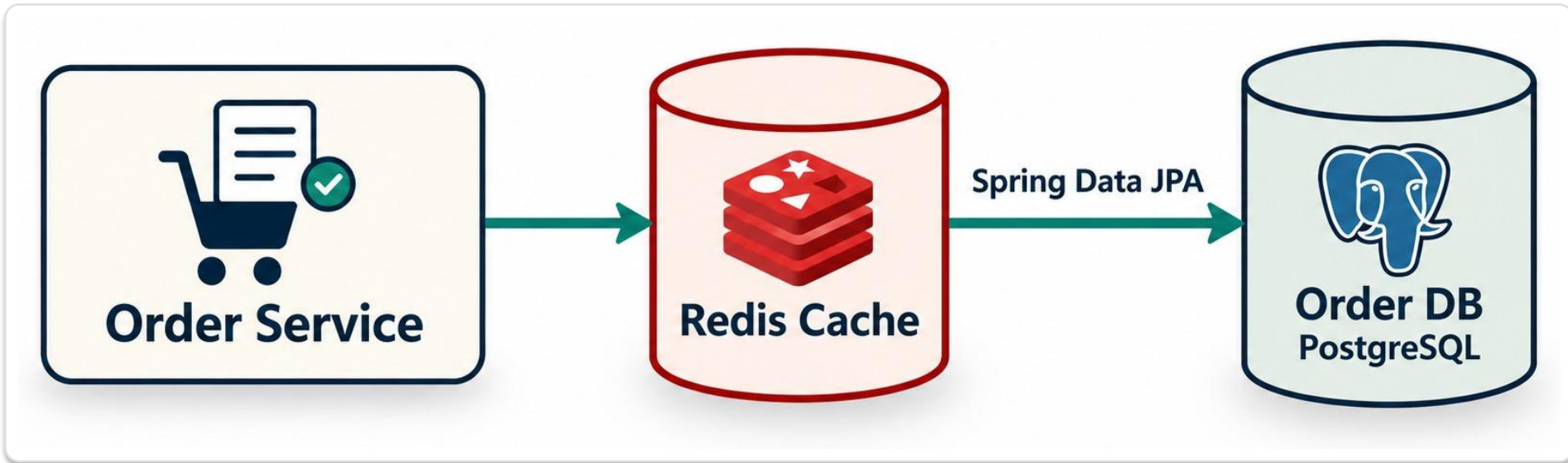
Reads are slow and repeated — how do we speed them up without serving wrong data?

IN THIS PART

- 1 Redis as an external cache
- 2 The cache-aside pattern
- 3 Expiration and invalidation
- 4 Caching in Spring Boot, in code
- 5 Exercise: what should we cache?
- 6 The complete data architecture

INTRODUCTION TO REDIS CACHING

Redis as an external in-memory store for frequently accessed data.



WHAT REDIS IS HERE

In-memory

Key-value

Shared by all pods

Not the system of record

Good candidates

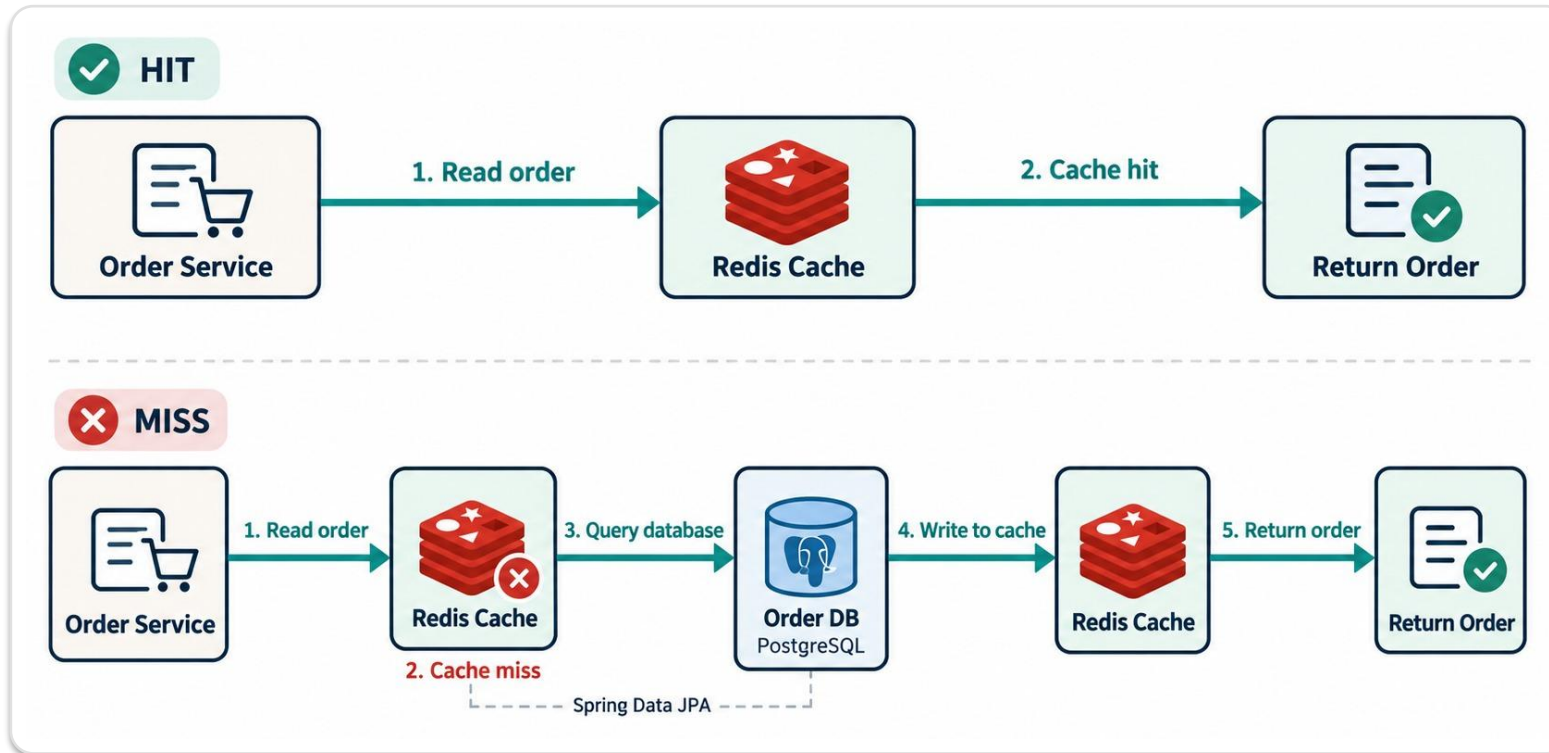
Product details, reference data, dashboard summaries — read often, change rarely.

WARNING A cache is a second copy of data — a consistency problem.

KEY TAKEAWAY Redis speeds up repeated reads — but every cache is a second copy to keep consistent.

CACHE-ASIDE PATTERN

Cache → miss → database → populate cache → next request hits.



What's stored

Key: order:1001
Value: {"id":1001,"status":"PAID"}
TTL: 600 seconds

Key design

Namespace by entity — order:1001, customer:501:profile. Cache DTOs, not entity graphs.

Cache down?

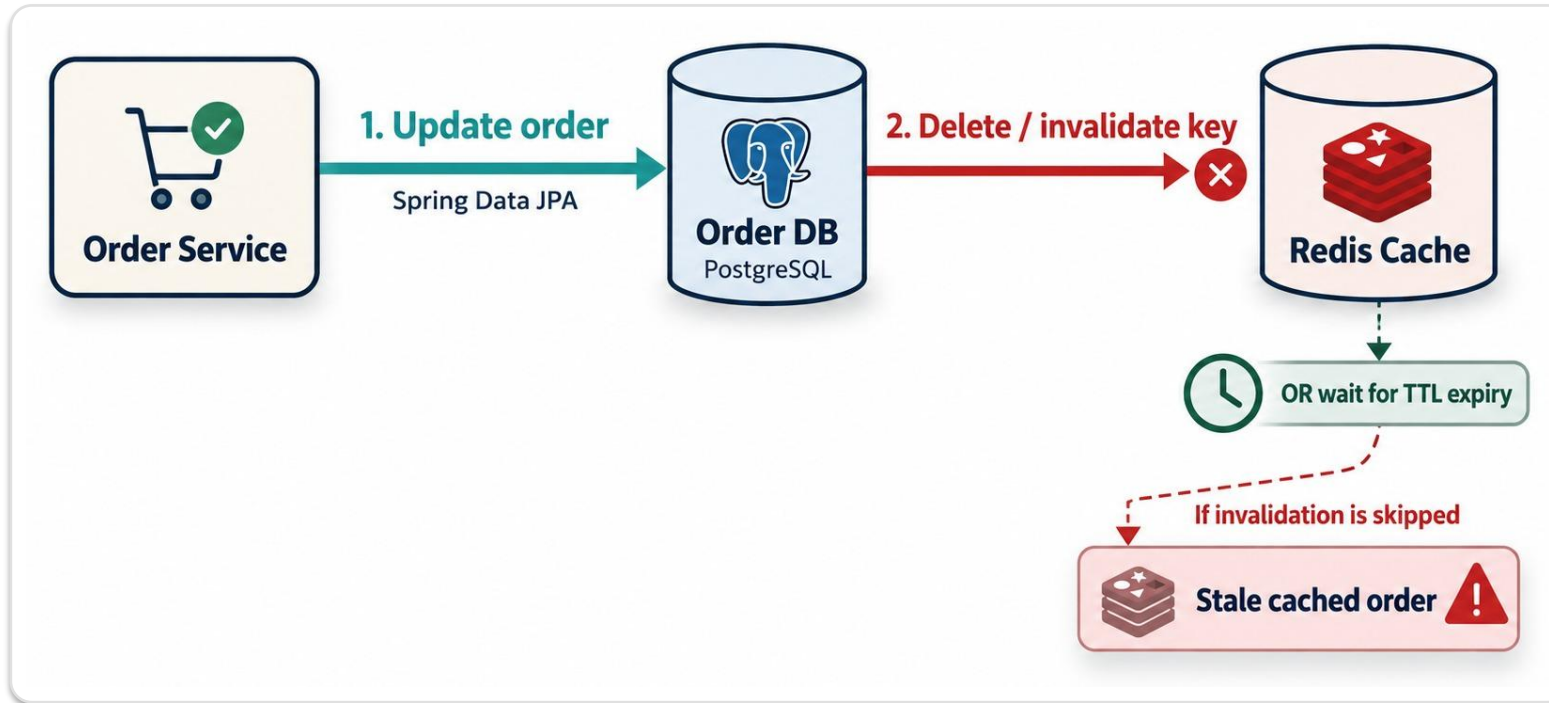
Fall back to the database — the service must still work.

OWNER The application decides what to cache and when.

KEY TAKEAWAY Check the cache first; on a miss, read the database and populate the cache for next time.

CACHE EXPIRATION, INVALIDATION AND CONSISTENCY

TTL, stale data, invalidation and cache/database consistency.



Long TTL

More cache hits, but staler data.

Short TTL

Fresher data, but more database traffic.

MATCH TTL TO THE DATA

Status: short

Profile: moderate

Products: long

Balances: don't

RULE Every cached read needs a matching evict on write.

KEY TAKEAWAY Caching improves performance by accepting extra state-management complexity.

CACHING IN SPRING BOOT

@Cacheable for cache-aside, @CacheEvict for invalidation, a TTL as the safety net.

✓ OrderService.java

```
@Cacheable(cacheNames = "orders", key = "#id")
public OrderResponse find(Long id) {
    return orders.findById(id)
        .map(OrderResponse::from)
        .orElseThrow(() -> new OrderNotFoundException(id));
}

@CacheEvict(cacheNames = "orders", key = "#id")
@Transactional
public OrderResponse cancel(Long id) { ... }
```

Enable Redis caching

```
@SpringBootApplication
@EnableCaching
public class OrderApplication { }

# application.yml
spring:
  data:
    redis:
      host: ${REDIS_HOST}
  cache:
    type: redis
    redis:
      time-to-live: 10m
```

@Cacheable

Checks Redis first; on a miss runs the method and stores the result.

@CacheEvict

Removes the entry when the order changes.

Transactions

Eviction can run before commit — a short TTL limits the stale window.

KEY TAKEAWAY Cache the read, evict on the write, and keep a TTL as the safety net.

EXERCISE: WHAT SHOULD WE CACHE?

Choose a caching strategy for each kind of data.

PRACTICE EXERCISE

10 MIN

SCENARIO

Decide the strategy for each item:

Product catalog page
Order status for tracking
Account balance
Exchange rates
Fraud decision
Branch and ATM list
Customer dashboard summary

YOUR TASKS

- 1 Cache it — or not?
- 2 If yes, choose a TTL and an invalidation trigger
- 3 Name one item where stale data causes real harm

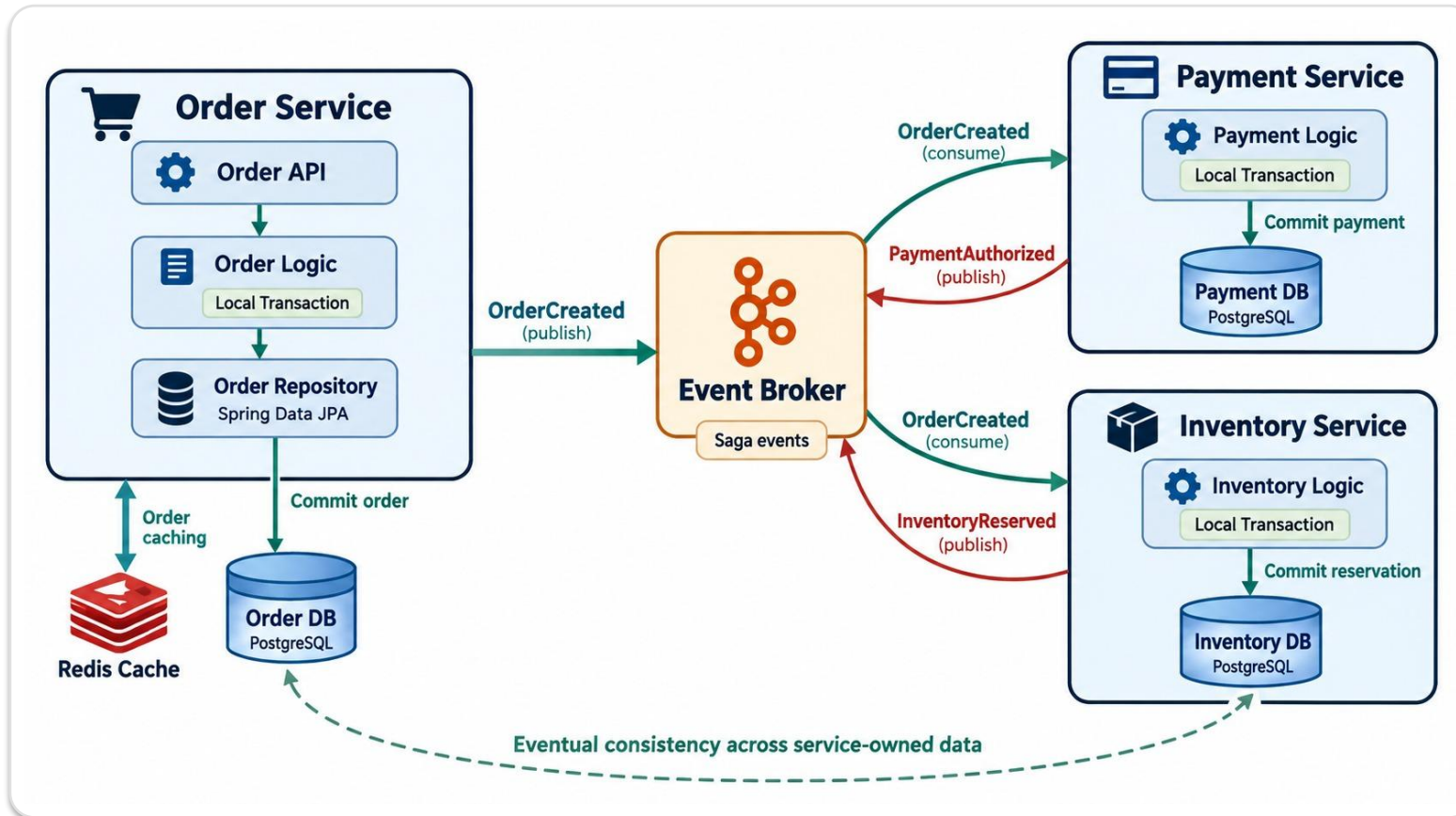
EXPECTED OUTCOME

- ✓ Catalog, branch list: cache, long TTL
- ✓ Exchange rates: cache for minutes
- ✓ Dashboard, order status: short TTL + evict on change
- ✓ Balance, fraud decision: don't cache
- ✓ A stale balance → a wrong spending decision

KEY TAKEAWAY Cache what is read often and tolerates staleness; never cache what must be exact right now.

END-TO-END MICROSERVICES DATA ARCHITECTURE

Service-owned SQL databases, local transactions, events and Sagas, and Redis caching.



EVERYTHING IN ONE PICTURE

- ✓ Service-owned databases
- ✓ Local transactions
- ✓ Saga events through the broker
- ✓ Eventual consistency
- ✓ Redis for fast reads

Read it

Follow **OrderCreated** out, and **PaymentAuthorized** and **InventoryReserved** back.

REMEMBER The database is the source of truth — not the cache, not the broker.

KEY TAKEAWAY Own your data, commit locally, coordinate with Sagas, and cache with care.

DATA MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that quietly couple services through their data.

MISCONCEPTION

REALITY

Database per service means a server each

It means ownership — schemas can share a server

Microservices can't use transactions

Local ACID transactions still work inside a service

A Saga rolls back like a database

It runs new, compensating business actions

ORM means you don't need SQL

Keys, indexes and N+1 queries still matter

The cache is the source of truth

The database is; the cache is a copy

ANTI-PATTERNS TO AVOID

- ✗ One database shared by services
- ✗ @ManyToOne into another service's data
- ✗ ddl-auto: update in production
- ✗ Returning entities from APIs
- ✗ Two-phase commit by default
- ✗ Caching balances without invalidation

REMEMBER Data is the hardest coupling — own a store, not a shared schema.

KEY TAKEAWAY Keep data owned, transactions local, compensations explicit and caches honest.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 Why should each service own its database or schema?

2 What does Spring Data JPA provide?

3 Why avoid JPA relationships across service boundaries?

4 What is the repository pattern for?

5 Where does @Transactional usually belong?

6 Why are distributed transactions hard?

7 What is a saga?

8 What is eventual consistency?

9 When is Redis useful?

10 Why should large list APIs use pagination?

MODULE 5 SUMMARY AND WHAT'S NEXT

From one service's SQL tables to a consistent, cached, multi-service data design.

YOU CAN NOW

- ✓ Persist data with Spring Data JPA, Flyway and a pooled DataSource
- ✓ Keep each service's data private — share through APIs and events
- ✓ Use local transactions and optimistic locking inside a service
- ✓ Coordinate cross-service work with Sagas and compensation
- ✓ Cache reads with Redis, with TTLs and invalidation

NEXT: MODULE 6 — RESILIENT, HIGH-PERFORMANCE SERVICES

- 1 What happens when a dependency slows down?
- 2 How do timeouts, retries and circuit breakers work together?
- 3 How do we find and fix performance bottlenecks?

BRING ALONG Your Lab 2 Transaction Service and its own transaction_db.

KEY TAKEAWAY Module 6 keeps these services responsive when dependencies slow down or fail.